

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра математической кибернетики и компьютерных наук

ЭТАПЫ КОМПИЛЯЦИИ ПРОГРАММ НА ЯЗЫКЕ C++
КУРСОВАЯ РАБОТА

студента 1 курса 111 группы
направления 02.03.02 — Фундаментальная информатика и информационные
технологии
факультета КНиИТ
Смирнова Сергея Дмитриевича

Научный руководитель
доцент, к. ф.-м. н.

Г. Г. Наркайтис

Заведующий кафедрой
доцент, к. ф.-м. н.

С. В. Миронов

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Математические основы компиляции	5
1.1 Регулярные выражения и автоматы	5
1.2 Контекстно-свободные грамматики	6
1.3 FIRST и FOLLOW	6
2 Архитектура компилятора	8
3 Программная реализация	10
4 Сравнение этапов	13
ЗАКЛЮЧЕНИЕ	14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15
Приложение А Полный код	16

ВВЕДЕНИЕ

Процесс компиляции программ является одной из ключевых составляющих современной программной инженерии. Он обеспечивает преобразование исходного кода, написанного на языках высокого уровня, в машинные инструкции, исполняемые вычислительной системой [1]. Без этапа компиляции невозможно представить разработку программного обеспечения, так как именно компилятор выполняет роль связующего звена между человеком и аппаратной частью компьютера.

Современные программные системы становятся всё более сложными, что приводит к увеличению требований к качеству и эффективности программного кода. В этих условиях особую значимость приобретает понимание внутренних механизмов работы компилятора. Знание этапов компиляции позволяет разработчикам писать более оптимизированный код, избегать типичных ошибок и лучше понимать сообщения об ошибках, выдаваемые компилятором.

Теоретической основой компиляции являются формальные языки и автоматы [2]. Лексический анализ основан на применении конечных автоматов, позволяющих эффективно разбивать входной текст программы на последовательность токенов. Синтаксический анализ, в свою очередь, опирается на аппарат контекстно-свободных грамматик, обеспечивающих проверку корректности структуры программы. Семантический анализ дополняет эти этапы, проверяя согласованность типов и корректность использования идентификаторов.

Язык программирования C++ широко используется в системном и прикладном программировании благодаря высокой производительности, гибкости и поддержке различных парадигм разработки [3]. Однако сложность языка и его богатые возможности делают особенно важным понимание процессов, происходящих на этапе компиляции.

Актуальность данной работы обусловлена необходимостью изучения фундаментальных принципов трансляции программ, которые лежат в основе работы современных компиляторов, таких как GCC и LLVM. Понимание этих принципов важно не только для программистов, но и для специалистов, занимающихся разработкой языков программирования, инструментов анализа кода и системного программного обеспечения.

Целью данной работы является изучение этапов компиляции программ на

языке С++ и их математических основ.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить основные этапы компиляции программ;
- рассмотреть математические модели, лежащие в основе лексического и синтаксического анализа;
- проанализировать архитектуру компилятора;
- реализовать простой лексический анализатор;
- исследовать особенности обработки программного кода на различных этапах компиляции.

Таким образом, исследование этапов компиляции и соответствующих математических моделей представляет значительный интерес как с теоретической, так и с практической точки зрения.

1 Математические основы компиляции

Компиляция программ опирается на строгие математические модели, позволяющие формально описывать структуру языков программирования и процессы их обработки. Основу этих моделей составляют теория формальных языков, теория автоматов и теория грамматик. Эти разделы дискретной математики позволяют не только описывать синтаксис языков, но и разрабатывать эффективные алгоритмы анализа программ.

1.1 Регулярные выражения и автоматы

На этапе лексического анализа исходный текст программы разбивается на элементарные единицы — токены. Для их описания используются регулярные выражения, представляющие собой формальный язык задания шаблонов строк.

Например, идентификаторы языка программирования могут быть заданы следующим регулярным выражением:

$$ID = [a-zA-Z_][a-zA-Z0-9_]* \quad (1.1)$$

Данное выражение означает, что идентификатор должен начинаться с буквы или символа подчёркивания, а затем может содержать произвольное количество букв, цифр и символов подчёркивания.

Регулярные выражения тесно связаны с конечными автоматами, которые используются для их реализации. Любому регулярному выражению можно сопоставить конечный автомат, распознающий соответствующий язык.

Конечный автомат определяется как пятёрка:

$$A = (Q, \Sigma, \delta, q_0, F) \quad (1.2)$$

где Q — конечное множество состояний, Σ — входной алфавит, δ — функция переходов, q_0 — начальное состояние, F — множество принимающих состояний.

В процессе работы лексического анализатора автомат последовательно обрабатывает символы входной строки, переходя между состояниями и определяя, принадлежит ли текущая последовательность символов некоторому токenu.

1.2 Контекстно-свободные грамматики

После лексического анализа выполняется синтаксический анализ, целью которого является проверка структурной корректности программы. Для этого используются контекстно-свободные грамматики.

Грамматика задаётся четвёркой:

$$G = (V, \Sigma, P, S) \quad (1.3)$$

где V — множество нетерминальных символов, Σ — множество терминалов, P — множество правил вывода, S — начальный символ.

Контекстно-свободные грамматики позволяют описывать вложенные структуры, такие как арифметические выражения, условные операторы и циклы.

Пример правила грамматики:

$$E \rightarrow E + T \mid T \quad (1.4)$$

Данное правило описывает арифметические выражения и отражает возможность рекурсивного построения выражений. На основе грамматик строятся синтаксические деревья (деревья разбора), которые отражают структуру программы и используются на последующих этапах компиляции.

1.3 FIRST и FOLLOW

При построении синтаксических анализаторов важную роль играют множества FIRST и FOLLOW, которые используются, например, при создании LL(1)-парсеров.

Множество FIRST определяет, какие терминальные символы могут появляться в начале строки, выводимой из данного символа:

$$\text{FIRST}(X) = \{a \mid X \Rightarrow^* a\beta\} \cup \{\varepsilon\} \quad (1.5)$$

Множество FOLLOW определяет, какие символы могут следовать непосредственно за данным нетерминалом в некоторой корректной строке языка:

$$\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \alpha A a \beta\} \quad (1.6)$$

Эти множества позволяют однозначно выбирать правила грамматики при разборе входной строки, что делает возможным построение эффективных син-

таксических анализаторов без возврата.

Например, при построении таблицы предсказательного анализа используются значения FIRST и FOLLOW для определения того, какое правило должно применяться в зависимости от текущего входного символа.

Таким образом, математические модели, лежащие в основе компиляции, обеспечивают формальную строгость и эффективность обработки программ, а также позволяют создавать надёжные и производительные компиляторы [4].

2 Архитектура компилятора

Архитектура компилятора представляет собой последовательность взаимосвязанных этапов, каждый из которых выполняет определённую функцию по преобразованию исходного кода программы в исполняемый машинный код. Несмотря на разнообразие существующих компиляторов, большинство из них реализуют схожую многоуровневую структуру обработки программы.

Основные этапы компиляции представлены на рисунке 2.1.

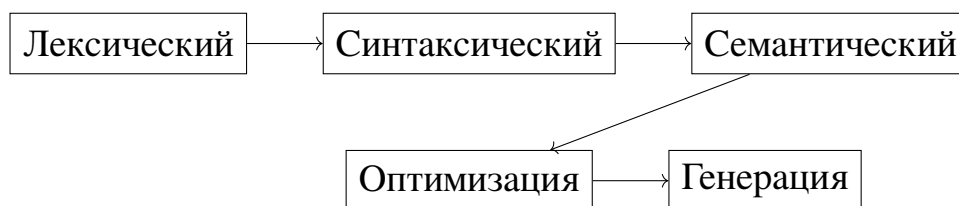


Рисунок 2.1 – Фазы компиляции

Как видно из рисунка 2.1, процесс компиляции можно разделить на несколько ключевых этапов.

Лексический анализ является первым этапом обработки программы. На данном этапе исходный текст разбивается на последовательность токенов — минимальных значимых единиц языка, таких как идентификаторы, ключевые слова, операторы и литералы. Результатом работы лексического анализатора является поток токенов, который передаётся на следующий этап.

Синтаксический анализ выполняет проверку правильности структуры программы. Используя правила грамматики, синтаксический анализатор строит дерево разбора, которое отражает вложенность конструкций программы. В случае обнаружения синтаксических ошибок (например, пропущенных скобок или неверного порядка операторов) компилятор сообщает об ошибке.

Семантический анализ отвечает за проверку корректности смысла программы. На этом этапе проверяются типы переменных, корректность объявлений и использования идентификаторов, а также соответствие операций их операндам. Результатом семантического анализа является аннотированное синтаксическое дерево, содержащее дополнительную информацию, необходимую для генерации кода.

Оптимизация направлена на улучшение характеристик программы без изменения её функциональности. Компилятор может выполнять различные виды оптимизаций, такие как удаление неиспользуемого кода, свёртка констант,

устранение избыточных вычислений и другие. Оптимизация позволяет повысить производительность и уменьшить объём исполняемого кода.

Генерация кода является завершающим этапом компиляции. На этом этапе формируется машинный или промежуточный код, который может быть выполнен процессором или передан на дальнейшую обработку (например, ассемблеру или линкеру). Качество генерации кода напрямую влияет на эффективность работы программы.

Дерево разбора (AST) представлено на рисунке 2.2.

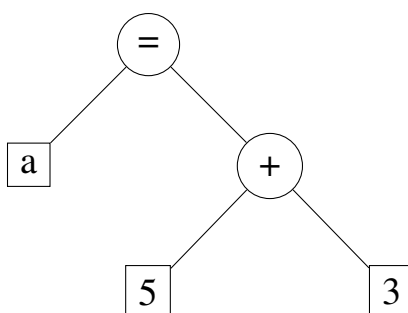


Рисунок 2.2 – Абстрактное синтаксическое дерево

Как показано на рисунке 2.2, абстрактное синтаксическое дерево (AST) представляет структуру программы в виде иерархии операций. Каждая вершина дерева соответствует некоторой операции, а её потомки — операндам. В отличие от полного дерева разбора, AST не содержит избыточной информации, связанной с синтаксическими деталями, такими как скобки или служебные символы.

Использование AST упрощает дальнейшие этапы компиляции, включая оптимизацию и генерацию кода, так как позволяет работать с более компактным и удобным представлением программы.

Таким образом, архитектура компилятора представляет собой последовательную цепочку преобразований, в ходе которой исходный текст программы постепенно преобразуется в эффективный исполняемый код. Чёткое разделение этапов обеспечивает модульность компилятора и позволяет разрабатывать и оптимизировать каждый этап независимо.

3 Программная реализация

Переменная `input` хранит исходный код, а `tokens` — результат анализа.

Реализация лексического анализатора представлена в листинге 1.

```
1  // Упрощённый лексический анализатор для C++-подобного языка
2  #include <iostream>
3  #include <vector>
4  #include <string>
5  #include <cctype>
6
7  using namespace std;
8
9  enum TokenType {
10     KEYWORD,    // int, float, return
11     IDENTIFIER, // a, b, result
12     NUMBER,     // 123, 3.14
13     OPERATOR,   // =, +, -, *, /
14     UNKNOWN
15 };
16
17 struct Token {
18     TokenType type;
19     string lexeme;
20 };
21
22 const vector<string> keywords = {"int", "float", "return"};
23
24 vector<Token> lex(const string& input) {
25     vector<Token> tokens;
26     size_t i = 0;
27     while (i < input.size()) {
28         char ch = input[i];
29         if (isspace(ch)) {
30             i++;
31             continue;
32         }
33         if (isalpha(ch) || ch == '_') {
34             string word;
35             while (i < input.size() && (isalnum(input[i]) || input[i] == '_'))
36                 ↪ {
37                 word += input[i];
38                 i++;
39             }
```

```

39         bool isKw = false;
40         for (const string& kw : keywords) {
41             if (word == kw) {
42                 tokens.push_back({KEYWORD, word});
43                 isKw = true;
44                 break;
45             }
46         }
47         if (!isKw) {
48             tokens.push_back({IDENTIFIER, word});
49         }
50         continue;
51     }
52     if (isdigit(ch)) {
53         string num;
54         while (i < input.size() && isdigit(input[i])) {
55             num += input[i];
56             i++;
57         }
58         tokens.push_back({NUMBER, num});
59         continue;
60     }
61     if (ch == '=' || ch == '+' || ch == '-' || ch == '*' || ch == '/') {
62         tokens.push_back({OPERATOR, string(1, ch)});
63         i++;
64         continue;
65     }
66     tokens.push_back({UNKNOWN, string(1, ch)});
67     i++;
68 }
69 return tokens;
70 }
71
72 void printTokens(const vector<Token>& tokens) {
73     for (const Token& t : tokens) {
74         string typeStr;
75         switch (t.type) {
76             case KEYWORD: typeStr = "KEYWORD"; break;
77             case IDENTIFIER: typeStr = "IDENTIFIER"; break;
78             case NUMBER: typeStr = "NUMBER"; break;
79             case OPERATOR: typeStr = "OPERATOR"; break;
80             case UNKNOWN: typeStr = "UNKNOWN"; break;

```

Listing 1: Лексический анализатор

Как видно из листинга 1, алгоритм выполняет посимвольный разбор строки и формирует токены.

Используются функции `isalpha` и `isdigit`, что упрощает анализ входных данных.

4 Сравнение этапов

Сравнение этапов компиляции приведено в таблице 4.1.

Таблица 4.1 – Фазы компиляции

Фаза	Вход	Ошибки
Лексический	текст	символы
Синтаксический	токены	грамматика
Семантический	AST	типы

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы были подробно рассмотрены основные этапы компиляции программ на языке C++, включая лексический, синтаксический и семантический анализ, а также этапы оптимизации и генерации кода. Показано, что процесс компиляции представляет собой сложную многоуровневую систему, основанную на строгих математических моделях. Были изучены теоретические основы компиляции, в частности теория формальных языков и автоматов, контекстно-свободные грамматики, а также методы построения синтаксических анализаторов. Рассмотренные модели позволяют формально описывать структуру языков программирования и обеспечивают корректность обработки исходного кода. В практической части работы был реализован упрощённый лексический анализатор, который демонстрирует процесс разбиения входной строки на токены. Полученные результаты подтверждают, что теоретические подходы, лежащие в основе компиляции, эффективно применяются при разработке программных инструментов. Также была рассмотрена архитектура компилятора и показана роль каждого этапа в процессе трансляции программ. Отмечено, что оптимизация кода играет важную роль в повышении эффективности выполнения программ. В результате выполненного исследования поставленная цель работы достигнута, а все поставленные задачи решены в полном объёме.

В дальнейшем данную работу можно расширить в следующих направлениях:

- разработка синтаксического анализатора;
- построение полного абстрактного синтаксического дерева;
- реализация генерации промежуточного и машинного кода;
- исследование и реализация методов оптимизации программ.

Таким образом, изучение этапов компиляции позволяет глубже понять принципы работы современных языков программирования и компиляторов, а также служит основой для разработки собственных систем трансляции.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Compilers: Principles, Techniques, and Tools [Text] / Aho, Alfred V., Lam, Monica S., Sethi, Ravi, and Ullman, Jeffrey D. — 2 ed. — Boston : Addison-Wesley, 2007.
- 2 Hopcroft, John E. Introduction to Automata Theory, Languages, and Computation [Text] / Hopcroft, John E., Motwani, Rajeev, and Ullman, Jeffrey D. — 3 ed. — [S. l.] : Pearson, 2006.
- 3 Stroustrup, Bjarne. The C++ Programming Language [Text]. — 4 ed. — [S. l.] : Addison-Wesley, 2013.
- 4 Grune, Dick. Parsing Techniques: A Practical Guide [Text] / Grune, Dick and Jacobs, Criel J.H. — 2 ed. — [S. l.] : Springer, 2012.
- 5 Aho, Alfred V. Principles of Compiler Design [Text] / Aho, Alfred V. and Ullman, Jeffrey D. — [S. l.] : Addison-Wesley, 1977.
- 6 Knuth, Donald E. The Art of Computer Programming, Volume 1: Fundamental Algorithms [Text]. — [S. l.] : Addison-Wesley, 1998.
- 7 Meyers, Scott. Effective Modern C++ [Text]. — [S. l.] : O'Reilly Media, 2014.
- 8 Muchnick, Steven S. Advanced Compiler Design and Implementation [Text]. — [S. l.] : Morgan Kaufmann, 1997.
- 9 Appel, Andrew W. Modern Compiler Implementation in C [Text]. — [S. l.] : Cambridge University Press, 2004.
- 10 cppreference.com. C++ reference [Electronic resource online]. — [S. l. : s. n.], 2024. — Access mode: <https://en.cppreference.com>. Accessed: 2026-05-17.
- 11 LLVM Project. LLVM Compiler Infrastructure [Electronic resource online]. — [S. l. : s. n.], 2024. — Access mode: <https://llvm.org>. Accessed: 2026-05-17.
- 12 GNU Project. GNU Compiler Collection (GCC) [Electronic resource online]. — [S. l. : s. n.], 2024. — Access mode: <https://gcc.gnu.org>. Accessed: 2026-05-17.

ПРИЛОЖЕНИЕ А

Полный код

```
1  // Упрощённый лексический анализатор для C++-подобного языка
2  #include <iostream>
3  #include <vector>
4  #include <string>
5  #include <cctype>
6
7  using namespace std;
8
9  enum TokenType {
10     KEYWORD,    // int, float, return
11     IDENTIFIER, // a, b, result
12     NUMBER,     // 123, 3.14
13     OPERATOR,   // =, +, -, *, /
14     UNKNOWN
15 };
16
17 struct Token {
18     TokenType type;
19     string lexeme;
20 };
21
22 const vector<string> keywords = {"int", "float", "return"};
23
24 vector<Token> lex(const string& input) {
25     vector<Token> tokens;
26     size_t i = 0;
27     while (i < input.size()) {
28         char ch = input[i];
29         if (isspace(ch)) {
30             i++;
31             continue;
32         }
33         if (isalpha(ch) || ch == '_') {
34             string word;
35             while (i < input.size() && (isalnum(input[i]) || input[i] == '_'))
36                 ↵ {
37                 word += input[i];
38                 i++;
39             }
40             bool isKw = false;
41             for (const string& kw : keywords) {
```



```

41         if (word == kw) {
42             tokens.push_back({KEYWORD, word});
43             isKw = true;
44             break;
45         }
46     }
47     if (!isKw) {
48         tokens.push_back({IDENTIFIER, word});
49     }
50     continue;
51 }
52 if (isdigit(ch)) {
53     string num;
54     while (i < input.size() && isdigit(input[i])) {
55         num += input[i];
56         i++;
57     }
58     tokens.push_back({NUMBER, num});
59     continue;
60 }
61 if (ch == '=' || ch == '+' || ch == '-' || ch == '*' || ch == '/') {
62     tokens.push_back({OPERATOR, string(1, ch)});
63     i++;
64     continue;
65 }
66 tokens.push_back({UNKNOWN, string(1, ch)});
67 i++;
68 }
69 return tokens;
70 }
71
72 void printTokens(const vector<Token>& tokens) {
73     for (const Token& t : tokens) {
74         string typeStr;
75         switch (t.type) {
76             case KEYWORD: typeStr = "KEYWORD"; break;
77             case IDENTIFIER: typeStr = "IDENTIFIER"; break;
78             case NUMBER: typeStr = "NUMBER"; break;
79             case OPERATOR: typeStr = "OPERATOR"; break;
80             case UNKNOWN: typeStr = "UNKNOWN"; break;
81         }
82         cout << "<" << typeStr << ", \">" << t.lexeme << "\">" << endl;
83     }
84 }

```

```
85
86  int main() {
87      string code = "int a = 5 + 3;";
88      cout << "Input: " << code << endl;
89      vector<Token> tokens = lex(code);
90      printTokens(tokens);
91      return 0;
92  }
```